

1. Objective

The purpose of this lab was to apply two clustering algorithms, K-Means and DBSCAN, to a cleaned YouTube comments dataset. The goal was to see how unsupervised learning can group comments into meaningful themes and to compare how both algorithms behave on real text data. This lab also helped me understand that clustering is not only about running code, but also about interpreting the results carefully.

2. Tools and Libraries Used

The following tools and libraries were used in this lab:

- Python 3.x
- pandas
- numpy
- matplotlib
- seaborn
- scikit-learn
 - TfidfVectorizer
 - KMeans
 - DBSCAN
 - PCA
 - StandardScaler

3. Dataset Description

For this lab, I used the file `cleaned_comments.csv`, which was created in Lab 2. The dataset contained cleaned YouTube comments and had the following columns:

- username
- timestamp_text
- comment_text
- cleaned_tokens

The dataset shape was: 739 comments and 4 columns

I loaded the dataset using `pandas.read_csv()` and checked the structure using `df.head()` and `df.shape`. I also checked the column names to make sure that a usable cleaned text column was present. In my file, the usable text column was `cleaned_tokens`, so I converted those token lists into normal text strings before applying TF-IDF.

Since my dataset already had the expected column names, I did not need to make major changes to the code structure.

4. Methodology / Procedure

4.1 Data Loading and Validation

First, I loaded the CSV file into a pandas DataFrame. After loading it, I printed the first few rows and checked the dataset shape to confirm that the file was read correctly. This step was important because clustering depends on having a properly cleaned and structured dataset. I confirmed that my dataset had 739 rows and contained the `cleaned_tokens` column needed for the next step.

4.2 Text Vectorization (TF-IDF)

Since machine learning algorithms cannot work directly with raw text, I converted the cleaned comments into numerical form using TF-IDF (Term Frequency–Inverse Document Frequency). The `cleaned_tokens` column contained tokenized words, so I joined the tokens into full strings before vectorization. Then I used:

- `TfidfVectorizer(max_features=2000)`

This created a TF-IDF matrix with shape: (739, 2000)

This means 739 comments were represented using up to 2000 text features.

4.3 K-Means Clustering

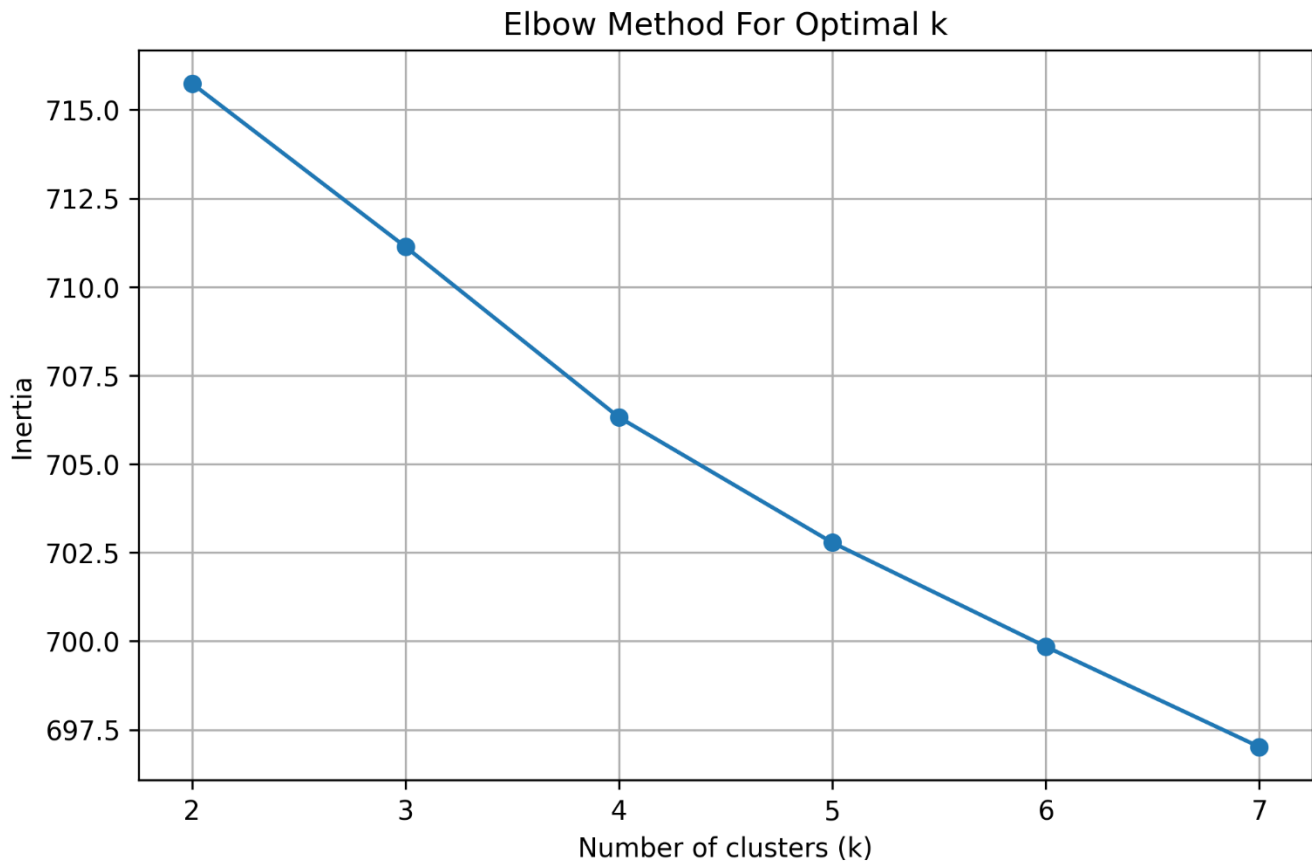
a. Elbow Method

To choose the number of clusters for K-Means, I used the elbow method. I tested values of k from 2 to 7 and recorded the inertia for each value. The inertia values were:

- $k = 2 \rightarrow 715.7451$
- $k = 3 \rightarrow 711.1335$
- $k = 4 \rightarrow 706.3198$
- $k = 5 \rightarrow 702.7815$
- $k = 6 \rightarrow 699.8443$
- $k = 7 \rightarrow 697.0154$

The decrease in inertia was gradual, so there was no very sharp elbow. Even so, I selected $k = 4$ because it looked like a reasonable middle point. It gave enough separation to explore different themes without creating too many very small clusters.

Figure 1: Elbow Method for Optimal k



b. K-Means Execution

After choosing $k = 4$, I ran the K-Means algorithm on the TF-IDF matrix and assigned a cluster label to each comment. The cluster sizes were:

- Cluster 0 → 431 comments
- Cluster 1 → 49 comments
- Cluster 2 → 114 comments
- Cluster 3 → 145 comments

The top keywords from each cluster were:

- **Cluster 0:** reply, like, video, space, know, one, brain, time, thing, channel
- **Cluster 1:** big, bang, universe, theory, infinite, happened, hole, like, black, time
- **Cluster 2:** infinite, universe, finite, space, there, amount, number, every, infinity, matter
- **Cluster 3:** universe, edge, billion, light, year, observable, time, like, reply, would

c. Interpretation

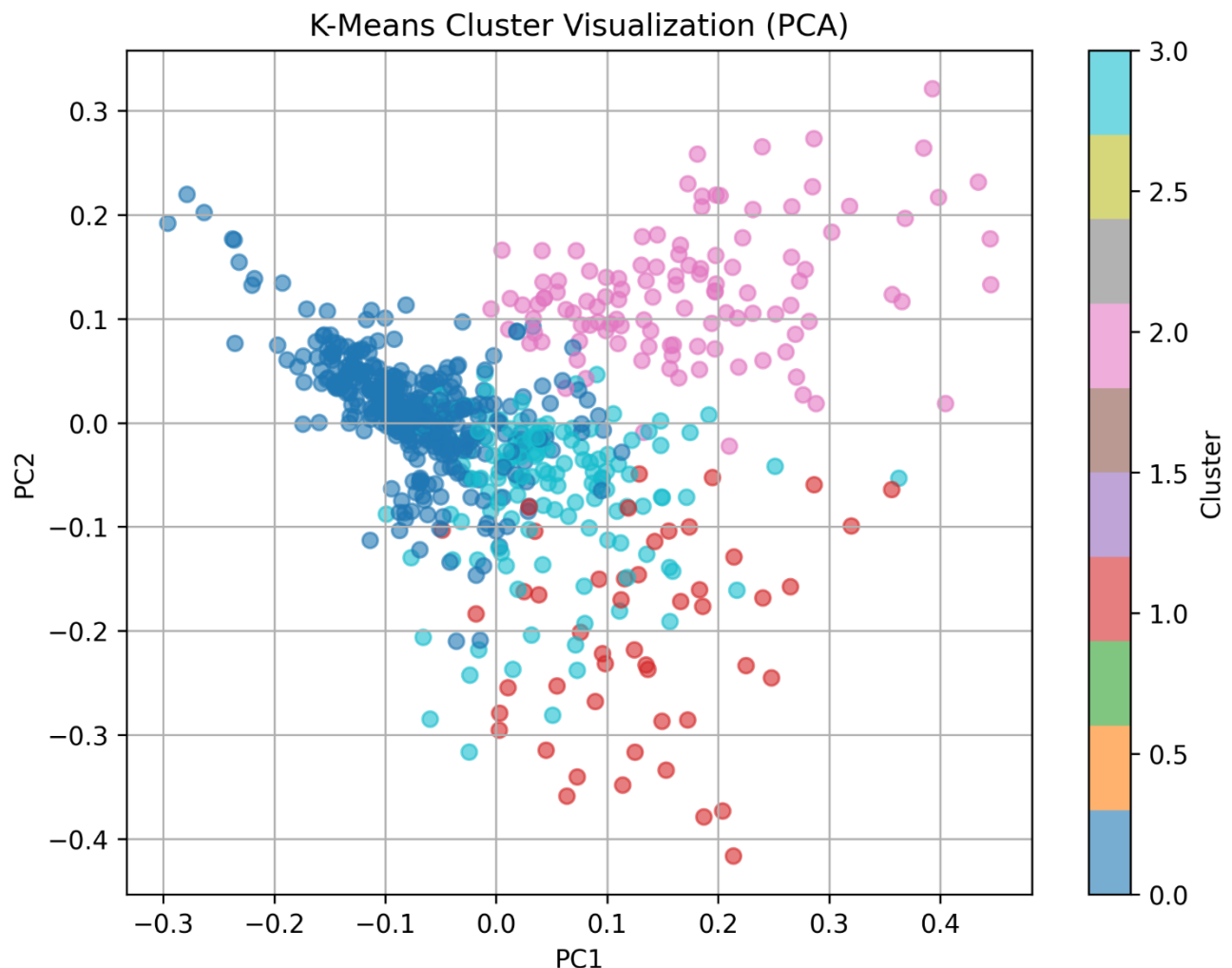
Some of the clusters were meaningful, while others were more mixed.

A good example was **Cluster 2**. Its keywords included words like *infinite*, *universe*, *finite*, *amount*, *number*, and *infinity*. The sample comments in this cluster were clearly about infinity, duplicate versions of people, and whether an infinite universe means all possibilities must exist. This cluster matched the main topic of the video very well.

A more confusing cluster was **Cluster 0**. It was much larger than the others and included broad words such as *reply*, *like*, *video*, *know*, and *thing*. The sample comments also showed that it mixed emotional reactions, side conversations, general replies, and jokes. Because of that, Cluster 0 did not represent one single clear theme as well as the other clusters.

To visualize the K-Means result, I reduced the TF-IDF data to two dimensions using PCA and plotted the cluster labels.

Figure 2: K-Means Cluster Visualization (PCA)



4.4 DBSCAN Clustering

a. Parameter Testing

For DBSCAN, I first scaled the sparse TF-IDF matrix using:

- StandardScaler(with_mean=False)

I then tested the required eps values from the lab instructions:

- eps = 0.3
- eps = 0.5
- eps = 0.7

The results were:

eps	min_samples	clusters	noise_points
0.3	5	0	739
0.5	5	0	739
0.7	5	0	739

These three values produced no clusters at all. Since all points became noise, I also tested a few larger values:

eps	min_samples	clusters	noise_points
5	5	0	739
10	5	0	739
20	5	1	702
30	5	1	557
40	5	1	330

Based on these results, I selected **eps = 40** for the final DBSCAN run, since it gave the strongest result among the tested values.

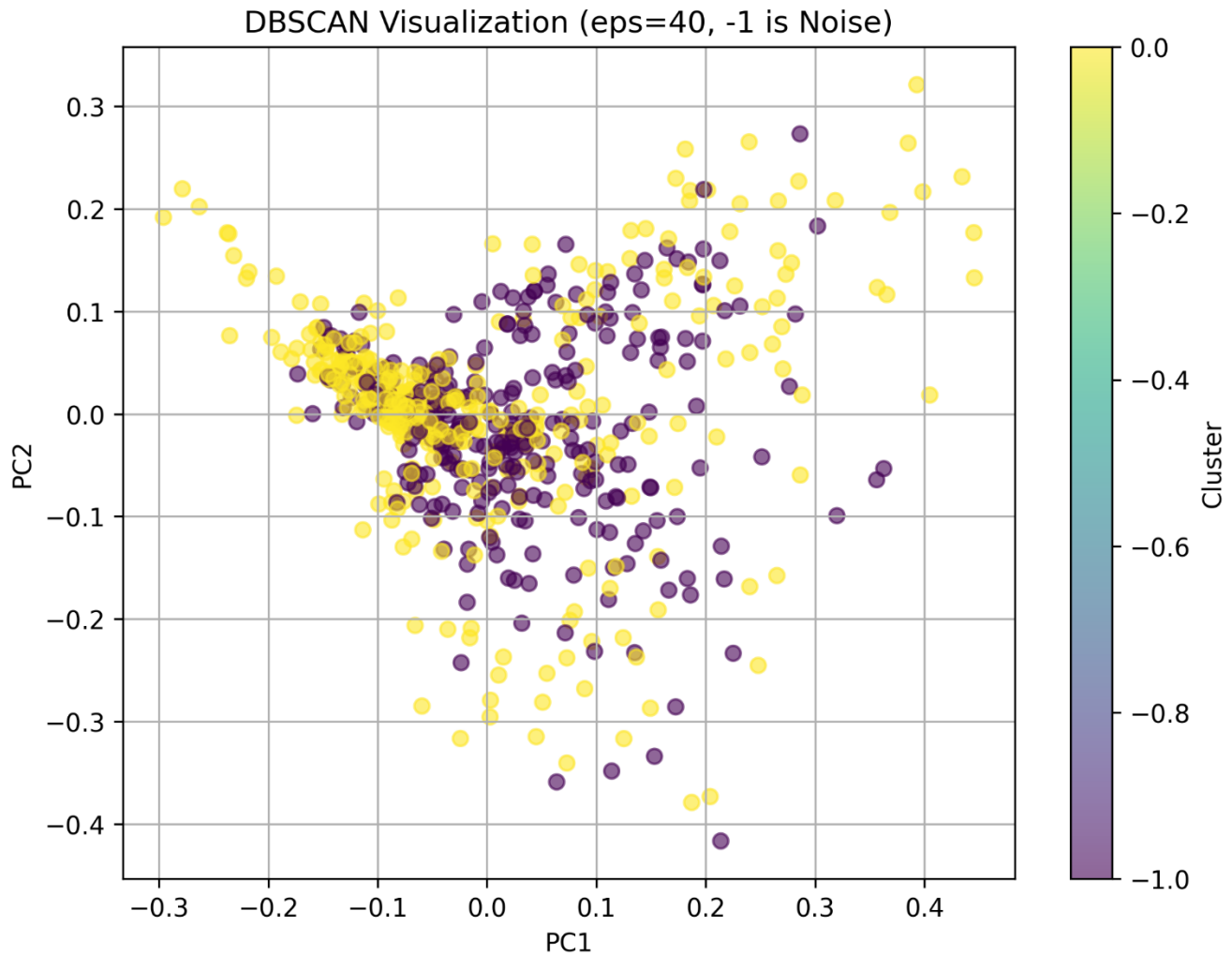
b. DBSCAN Visualization

With eps = 40, DBSCAN produced:

- 1 cluster
- 330 noise points

This means a large number of comments still did not belong to any dense region. I used the same PCA coordinates from the K-Means section to visualize the DBSCAN output.

Figure 3: DBSCAN Visualization (eps = 40, -1 is Noise)



c. Interpretation

DBSCAN did not perform as well as K-Means on this dataset. Even after increasing ϵ , it still found only one cluster and marked many comments as noise. This suggests that the comments are spread out in TF-IDF space and do not form many dense text neighborhoods.

That result is understandable because YouTube comments are often short, varied, informal, and sometimes very unique in wording. DBSCAN works best when the data has clear dense regions, but that was not strongly present here.

4.5 Comparison of Algorithms

K-Means and DBSCAN behaved very differently on this dataset.

Interpretability

- K-Means was easier to interpret because it assigned every comment to a cluster. This made it easier to inspect cluster keywords and understand broad discussion themes.
- DBSCAN was harder to interpret because it produced only one cluster and a large number of noise points.

Number of Clusters

- K-Means produced 4 clusters, which gave a more detailed view of the comment themes.
- DBSCAN produced only 1 cluster at the chosen setting, which gave much less thematic separation.

Noise Handling

- K-Means does not detect noise and forces every point into a cluster.
- DBSCAN is better at handling noise, and in this case it showed that many comments did not belong to a dense group. That is useful information, but it also meant less meaningful grouping overall.

Meaningful Insight Extraction

For this dataset, K-Means extracted more useful insight because it separated comments into broader themes such as infinity, the Big Bang, the observable universe, and general reactions. DBSCAN mainly showed that the data was too scattered for strong density-based grouping.

5. Results and Discussion

The following outputs were produced during the lab:

- Elbow plot for K-Means
- Top keywords for each K-Means cluster
- PCA visualization for K-Means
- DBSCAN results table for multiple eps values
- PCA visualization for DBSCAN
- Final clustered CSV file

For my dataset, **K-Means performed better than DBSCAN.**

The K-Means clusters showed several understandable themes. For example:

- comments about infinity and exact copies

- comments about the Big Bang and finite versus infinite universe
- comments about the observable edge of the universe
- general reactions and side conversations

One interesting result was that Cluster 2 was clearly focused on the idea of infinity, while Cluster 0 was much broader and mixed. This shows that clustering can reveal both strong and weak thematic groups in the same dataset.

The biggest weakness I observed was with DBSCAN. Even with larger `eps` values, it only formed one cluster and still marked many comments as noise. That tells me that not every clustering algorithm works equally well on every type of text data.

A strength of K-Means is that it gives a clearer overall summary of the discussion. A weakness is that it forces all comments into clusters, even when some comments may not clearly belong anywhere.

A strength of DBSCAN is that it can identify outliers and noise. A weakness is that it is very sensitive to parameter choices and may struggle with sparse, high-dimensional text data like TF-IDF comment vectors.

6. Critical Prompts

1. Initial hypothesis about cluster quality

My dataset is fairly large, so I expected the clustering to show more meaningful themes. A larger number of comments usually gives the algorithms more repeated patterns to group together.

2. Choice of `k` and justification

I chose `k = 4` because the elbow plot showed a gradual decrease in inertia, and 4 seemed like a reasonable balance between too few and too many clusters. The elbow was not very sharp, so the choice was somewhat subjective, but 4 gave a practical grouping for this dataset. A good cluster was **Cluster 2** because its top words and sample comments clearly focused on infinity, copies, and the idea of an infinite universe. A more confusing cluster was **Cluster 0** because it was very large and mixed general reactions, replies, and side discussions instead of one clear topic.

3. K-Means vs DBSCAN: which was more useful and why?

K-Means gave me a more useful understanding of the comments because it assigned every comment to a cluster and made it easier to identify broad discussion themes in the video. DBSCAN marked many comments as noise, which suggests that the comments were too spread out in TF-IDF space to form dense groups. DBSCAN would be more useful in a situation where the comments contain many outliers, spam, or small irregular groups that do not fit well into a fixed number of clusters.

4. How dataset characteristics influenced the results, and the key lesson for real-world applications

My dataset had 739 comments, which is large enough for clustering and gives enough repeated patterns for analysis. However, the topic and language style of YouTube comments still affected the results because the comments varied in length, tone, wording, and focus. That made K-Means better for finding broad themes, while DBSCAN struggled because many comments did not form dense neighborhoods. The most important lesson for real-world use is that clustering can reveal useful patterns, but the results must always be interpreted carefully in context instead of being accepted blindly.

7. Conclusion

In this lab, I applied K-Means and DBSCAN clustering to a cleaned YouTube comments dataset and compared their performance. The results showed that K-Means was more effective for this dataset because it produced several understandable clusters and made it easier to identify broad themes in the discussion. DBSCAN was less effective because it found only one cluster and marked many comments as noise.

This lab showed that clustering text data can be useful, but the output depends heavily on the nature of the dataset and the algorithm used. It also showed that interpretation is just as important as the code itself. A clustering result is only useful when it is examined carefully and connected back to the actual meaning of the comments.